

Task 1: The Problem

Run a Postgres or MySQL container

Create some data inside it (a table, a few rows — anything)

```
ubuntu@ip-172-31-30-155:~$ docker exec -it postgres-db psql -U myuser -d mydatabase
psql (15.17 (Debian 15.17-1.pgdg13+1))
Type "help" for help.

mydatabase=# CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email TEXT NOT NULL,
    age INTEGER CHECK (age > 18),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE
mydatabase=# select table from mydatabase;
ERROR:  syntax error at or near "table"
LINE 1: select table from mydatabase;
           ^
mydatabase=# \l
mydatabase=# \l

               List of databases
  Name | Owner | Encoding | Collate | Ctype | ICU Locale | Locale Provider | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----
mydatabase | myuser | UTF8 | en_US.utf8 | en_US.utf8 |  | libc | 
postgres | myuser | UTF8 | en_US.utf8 | en_US.utf8 |  | libc | 
template0 | myuser | UTF8 | en_US.utf8 | en_US.utf8 |  | libc | =c/myuser +
template1 | myuser | UTF8 | en_US.utf8 | en_US.utf8 |  | libc | myuser=CTC/myuser +
         =c/myuser +
         myuser=CTC/myuser
(4 rows)

mydatabase=# \dt
               List of relations
 Schema | Name | Type | Owner
-----+-----+-----+-----
public | users | table | myuser
(1 row)

mydatabase=#
```

Stop and remove the container

```
ubuntu@ip-172-31-30-155:~$ docker stop postgres-db && docker rm postgres-db
postgres-db
postgres-db
ubuntu@ip-172-31-30-155:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
ubuntu@ip-172-31-30-155:~$
```

Run a new one — is your data still there?

```
ubuntu@ip-172-31-30-155:~$ docker run --name postgres-db -e POSTGRES_PASSWORD=Test@123 -e POSTGRES_USER=myuser -e POSTGRES_DB=mydatabas
e -d postgres:15.17-trixie
1e0cb2e9eea798e038383b242b53cac8d7f079d5acd52a5b528e8bd2dd5526c6
ubuntu@ip-172-31-30-155:~$ docker exec -it postgres-db psql -U myuser -d mydatabase
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL:  role "myuser" does not exist
ubuntu@ip-172-31-30-155:~$ docker exec -it postgres-db psql -U myuser -d mydatabase
psql (15.17 (Debian 15.17-1.pgdg13+1))
Type "help" for help.

mydatabase=# /dt
mydatabase=# /d
mydatabase=# \l

               List of databases
  Name | Owner | Encoding | Collate | Ctype | ICU Locale | Locale Provider | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----
mydatabase | myuser | UTF8 | en_US.utf8 | en_US.utf8 |  | libc | 
postgres | myuser | UTF8 | en_US.utf8 | en_US.utf8 |  | libc | 
template0 | myuser | UTF8 | en_US.utf8 | en_US.utf8 |  | libc | =c/myuser +
template1 | myuser | UTF8 | en_US.utf8 | en_US.utf8 |  | libc | myuser=CTC/myuser +
         =c/myuser +
         myuser=CTC/myuser
(4 rows)

mydatabase=# \dt
Did not find any relations.
mydatabase=#
```

Write what happened and why.

All the data in the database is lost , because the database was not mapped to a volume and all the data associated with the container is deleted when the container is removed.

Task 2: Named Volumes

Create a named volume

```
ubuntu@ip-172-31-30-155:~$ docker volume ls
DRIVER      VOLUME NAME
local       2b9cfa060352041b94748011418beb941a15a2086ab226125b4212e4615d3820
local       37a573903ca13aaf37e79e18efa4363f99a415d4b74bb3ac514642406161b34f
local       f354068af980eaa81417e37a37e56a8dcffa8689beaceb8b29b027b021348b53
ubuntu@ip-172-31-30-155:~$ docker volume create postgres-data
postgres-data
ubuntu@ip-172-31-30-155:~$ docker volume ls
DRIVER      VOLUME NAME
local       2b9cfa060352041b94748011418beb941a15a2086ab226125b4212e4615d3820
local       37a573903ca13aaf37e79e18efa4363f99a415d4b74bb3ac514642406161b34f
local       f354068af980eaa81417e37a37e56a8dcffa8689beaceb8b29b027b021348b53
local       postgres-data
ubuntu@ip-172-31-30-155:~$ |
```

```
ubuntu@ip-172-31-30-155:~$ docker volume inspect postgres-data
[
  {
    "CreatedAt": "2026-03-16T11:25:24Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/postgres-data/_data",
    "Name": "postgres-data",
    "Options": null,
    "Scope": "local"
  }
]
ubuntu@ip-172-31-30-155:~$
```

Run the same database container, but this time attach the volume to it

```
ubuntu@ip-172-31-30-155:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-30-155:~$ docker run --name postgres-db -e POSTGRES_PASSWORD=Test@123 -e POSTGRES_USER=myuser -e POSTGRES_DB=mydatabase -v postgres-data:/var/lib/postgresql/data -d postgres:15.17-trixie
613d2939ddf47b86001a4c79f9edd34757eb615dd99a2b5b5bd91149180d199a
ubuntu@ip-172-31-30-155:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
613d2939ddf4   postgres:15.17-trixie   "docker-entrypoint.s..."   3 seconds ago   Up 3 seconds   5432/tcp   postgres-db
ubuntu@ip-172-31-30-155:~$ docker exec -it postgres-db psql -U myuser -d mydatabase
psql (15.17 (Debian 15.17-1.pgdg13+1))
Type "help" for help.

mydatabase=# \d
Did not find any relations.
mydatabase=# \dt
Did not find any relations.
mydatabase=# \l
                                List of databases
  Name | Owner  | Encoding | Collate | Ctype  | ICU Locale | Locale Provider | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----
mydatabase | myuser | UTF8     | en_US.utf8 | en_US.utf8 |              | libc            |
postgres  | myuser | UTF8     | en_US.utf8 | en_US.utf8 |              | libc            |
template0 | myuser | UTF8     | en_US.utf8 | en_US.utf8 |              | libc            | =c/myuser      +
           |        |          |            |            |              |                 | myuser=Ctc/myuser +
template1 | myuser | UTF8     | en_US.utf8 | en_US.utf8 |              | libc            | =c/myuser      +
           |        |          |            |            |              |                 | myuser=Ctc/myuser +
(4 rows)

mydatabase=# CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email TEXT NOT NULL,
    age INTEGER CHECK (age > 18),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE
```

Add some data, stop and remove the container

```
mydatabase=# \dt
                                List of relations
 Schema | Name  | Type  | Owner
-----+-----+-----+-----
 public | users | table | myuser
(1 row)

mydatabase=#
```

Run a brand new container with the same volume

```
ubuntu@ip-172-31-30-155:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
613d2939ddf4   postgres:15.17-trixie   "docker-entrypoint.s..."   3 minutes ago   Up 3 minutes   5432/tcp   postgres-db
ubuntu@ip-172-31-30-155:~$ docker stop postgres-db && docker rm postgres-db
postgres-db
postgres-db
ubuntu@ip-172-31-30-155:~$ docker ps -a
unknown shorthand flag: 'A' in -a

Usage:  docker ps [OPTIONS]

Run 'docker ps --help' for more information
ubuntu@ip-172-31-30-155:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-30-155:~$
```

```
ubuntu@ip-172-31-30-155:~$ docker run --name postgres-db-1 -e POSTGRES_PASSWORD=Test@123 -e POSTGRES_USER=myuser -e POSTGRES_DB=mydatabase -v postgres-data:/var/lib/postgresql/data -d postgres:15.17-trixie
e6345d65b7bdc54af441601d1e97cbf15f0c09ec136d1cd2b9a6d77dd4b87ea5
ubuntu@ip-172-31-30-155:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
e6345d65b7bd   postgres:15.17-trixie   "docker-entrypoint.s..."   4 seconds ago   Up 4 seconds   5432/tcp   postgres-db-1
ubuntu@ip-172-31-30-155:~$
```

Is the data still there? Yes

```
ubuntu@ip-172-31-30-155:~$ docker run --name postgres-db-1 -e POSTGRES_PASSWORD=Test@123 -e POSTGRES_USER=myuser -e POSTGRES_DB=mydatabase -v postgres-data:/var/lib/postgresql/data -d postgres:15.17-trixie
e6345d65b7bdc54af441601d1e97cbf15f0c09ec136d1cd2b9a6d77dd4b87ea5
ubuntu@ip-172-31-30-155:~$ docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS        PORTS          NAMES
e6345d65b7bd   postgres:15.17-trixie "docker-entrypoint.s..." 4 seconds ago   Up 4 seconds   5432/tcp       postgres-db-1
ubuntu@ip-172-31-30-155:~$ docker exec -it postgres-db-1 psql -U myuser -d mydatabase
psql (15.17 (Debian 15.17-1.pgdg13+1))
Type "help" for help.

mydatabase=# \l
              List of databases
  Name | Owner | Encoding | Collate | Ctype | ICU Locale | Locale Provider | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----
mydatabase | myuser | UTF8     | en_US.utf8 | en_US.utf8 |           | libc             |
postgres  | myuser | UTF8     | en_US.utf8 | en_US.utf8 |           | libc             |
template0 | myuser | UTF8     | en_US.utf8 | en_US.utf8 |           | libc             | =C/myuser, myuser=CTC/myuser,
template1 | myuser | UTF8     | en_US.utf8 | en_US.utf8 |           | libc             | =C/myuser, myuser=CTC/myuser
(4 rows)

mydatabase=# \dt
              List of relations
 Schema | Name | Type | Owner
-----+-----+-----+-----
 public | users | table | myuser
(1 row)

mydatabase=#
```

Task 3: Bind Mounts

Create a folder on your host machine with an index.html file

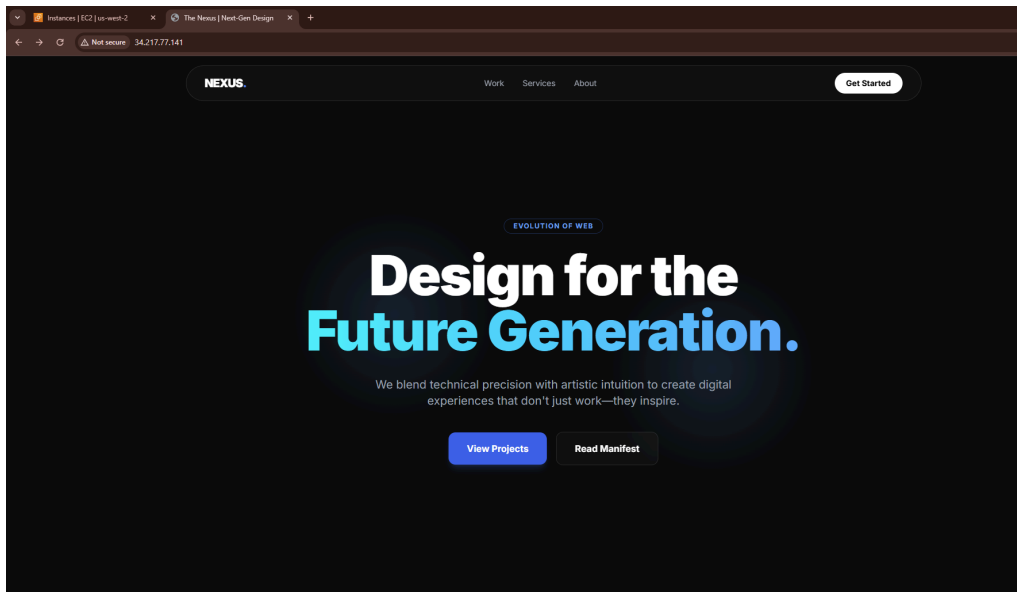
```
root@ip-172-31-30-155:/var/www/html# vi index.html
root@ip-172-31-30-155:/var/www/html# ls -ltr
total 12
-rw-r--r-- 1 root root 615 Mar  5 13:43 index.nginx-debian.html
-rw-r--r-- 1 root root 4513 Mar 16 12:03 index.html
root@ip-172-31-30-155:/var/www/html#
```

Run an Nginx container and bind mount your folder to the Nginx web directory

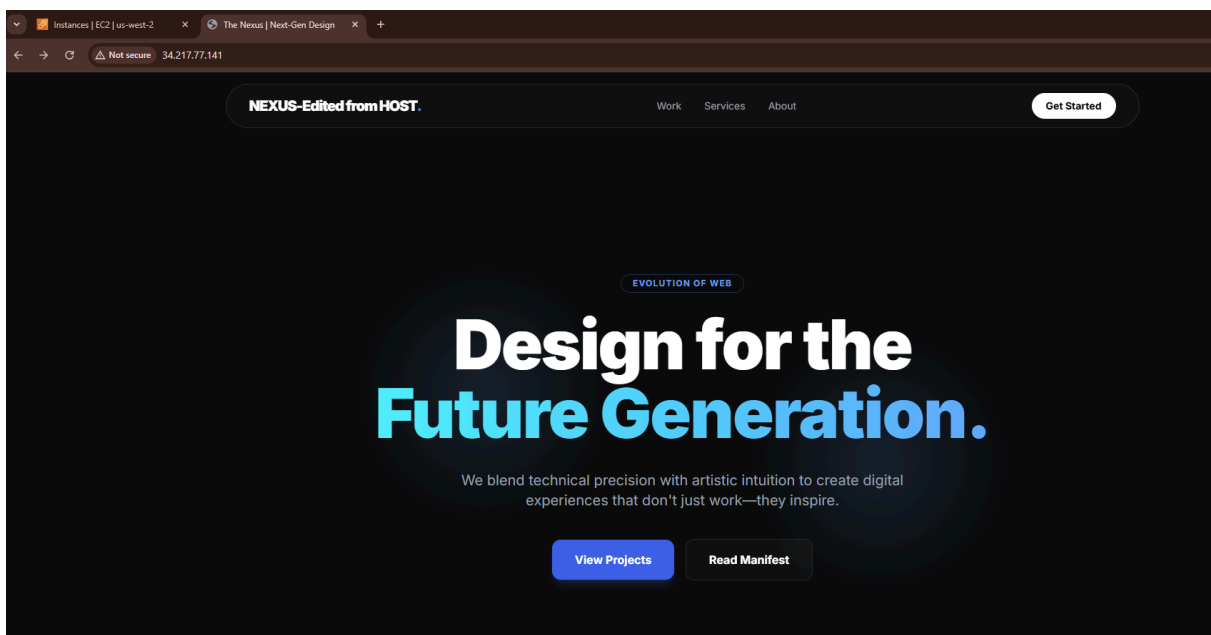
```
root@ip-172-31-30-155:/var/www/html# docker run --name nginx-server -d -p 80:80 -v /var/www/html:/usr/share/nginx/html nginx
```

```
root@ip-172-31-30-155:/var/www/html# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
a0c81654c093   nginx     "/docker-entrypoint...." 16 seconds ago   Up 16 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp   nginx-server
root@ip-172-31-30-155:/var/www/html#
```

Access the page in your browser



Edit the index.html on your host — refresh the browser



Write in your notes: What is the difference between a named volume and a bind mount?

A named volume is created and maintained by docker

A bind mount is a user-defined volume which needs to be maintained by the user

Feature	Named Volume	Bind Mount
Host Location	Managed by Docker (Hidden)	Anywhere you choose (Visible)
Ease of Use	High (Docker handles permissions)	Moderate (You must manage permissions)
Portability	High (Easy to back up/move)	Low (Paths change between Mac/Win/Linux)
Performance	Excellent	Good (But can be slow on Mac/Windows)

When to choose which?

- **Use a Bind Mount** when you are **coding**. You want your code changes to reflect instantly without rebuilding the image.(eg.index.html on nginx server)
- **Use a Named Volume** for your **Database** (Postgres/MySQL). It's safer, faster, and Docker ensures the data is isolated from your other files.

Task 4: Docker Networking Basics

List all Docker networks on your machine

```
ubuntu@ip-172-31-30-155:~$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
d26c820e9944        bridge             bridge              local
85493ae0b009        host               host                local
e1462d75f42c        none               null                local
ubuntu@ip-172-31-30-155:~$
```

Inspect the default bridge network

```
ubuntu@ip-172-31-30-155:~$ docker network inspect bridge
[
  {
    "Name": "bridge",
    "Id": "d26c820e9944104a0fbde3b64ffc3f23652dd4d65442f9e23d4e8695497a0acb",
    "Created": "2026-03-16T09:40:33.262227224Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.17.0.0/16",
          "Gateway": "172.17.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "a0c81654c093a8e928b258f8c18998587a9b2445e84c3530aa716a298095e4be": {
        "Name": "nginx-server",
        "EndpointID": "a4fb79832a44efdcce01d8a7a7494753a52c6738f62452fa27aa77c09ee77fe4",
        "MacAddress": "ba:2e:82:6a:3f:77",
        "IPv4Address": "172.17.0.2/16",
        "IPv6Address": ""
      }
    },
    "Options": {
      "com.docker.network.bridge.default_bridge": "true",
      "com.docker.network.bridge.enable_icc": "true",
      "com.docker.network.bridge.enable_ip_masquerade": "true",

```

```

ubuntu@ip-172-31-30-155:~$ docker network inspect bridge
[
  {
    "Name": "bridge",
    "Id": "d26c820e9944104a0fbde3b64ffc3f23652dd4d65442f9e23d4e8695497a0acb",
    "Created": "2026-03-16T09:40:33.262227224Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.17.0.0/16",
          "Gateway": "172.17.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "15fa9ff2c10905fd7f0b5eedd9bb3cf1ddec091b9f564a696190bfc6b8ef0c04": {
        "Name": "nginx-2",
        "EndpointID": "778fe9c4ad2dbf77a99e3e5071924728f817f0f0a9a72f9590ccd37cb8abfd5f",
        "MacAddress": "f6:a2:2f:ba:2d:7f",
        "IPv4Address": "172.17.0.3/16",
        "IPv6Address": ""
      },
      "1ac56b11ccbd65e1693527733a453d8d3e2f34a99eebecdb6450deb43bc610aa": {
        "Name": "nginx-1",
        "EndpointID": "b750abbe8be2784e427cdaa6ef9c220ae847de22299da7dab62f416f0236cf51",
        "MacAddress": "ae:36:fa:5f:58:bf",
        "IPv4Address": "172.17.0.2/16",
        "IPv6Address": ""
      }
    },
    "Options": {
      "com.docker.network.bridge.default_bridge": "true",
      "com.docker.network.bridge.enable_icc": "true",
      "com.docker.network.bridge.enable_ip_masquerade": "true",
      "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
      "com.docker.network.bridge.name": "docker0",
      "com.docker.network.driver.mtu": "1500"
    },
    "Labels": {}
  }
]

```

```

ubuntu@ip-172-31-30-155:~$ docker exec nginx-1 ping nginx-2
ping: bad address 'nginx-2'
ubuntu@ip-172-31-30-155:~$ docker exec nginx-1 ping 172.17.0.3
PING 172.17.0.3 (172.17.0.3): 56 data bytes
64 bytes from 172.17.0.3: seq=0 ttl=64 time=0.084 ms
64 bytes from 172.17.0.3: seq=1 ttl=64 time=0.062 ms
64 bytes from 172.17.0.3: seq=2 ttl=64 time=0.073 ms
64 bytes from 172.17.0.3: seq=3 ttl=64 time=0.065 ms
64 bytes from 172.17.0.3: seq=4 ttl=64 time=0.072 ms
64 bytes from 172.17.0.3: seq=5 ttl=64 time=0.059 ms
64 bytes from 172.17.0.3: seq=6 ttl=64 time=0.063 ms
^Cubuntu@ip-172-31-30-155:~$ docker exec 172.17.0.2 ping 172.17.0.3
Error response from daemon: No such container: 172.17.0.2
ubuntu@ip-172-31-30-155:~$ docker exec nginx-2 ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2): 56 data bytes
64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.082 ms
64 bytes from 172.17.0.2: seq=1 ttl=64 time=0.073 ms
64 bytes from 172.17.0.2: seq=2 ttl=64 time=0.075 ms
64 bytes from 172.17.0.2: seq=3 ttl=64 time=0.150 ms
ubuntu@ip-172-31-30-155:~$

```


Run two containers on the default bridge — can they ping each other by name?

No , unable to ping each other using names

Run two containers on the default bridge — can they ping each other by IP?

Yes , able to ping each other by their IP address.

Task 5: Custom Networks

Create a custom bridge network called my-app-net

```
ubuntu@ip-172-31-30-155:~$ docker network create my-app-net --driver bridge
b16516e3439a7c8b68472138bf4f81373ca689ef2bc0f2d91a1c4286fe299ead
ubuntu@ip-172-31-30-155:~$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
d26c820e9944        bridge              bridge              local
85493ae0b009        host                host                local
b16516e3439a        my-app-net          bridge              local
e1462d75f42c        none                null                local
ubuntu@ip-172-31-30-155:~$
```

Run two containers on my-app-net

```
ubuntu@ip-172-31-30-155:~$ docker network inspect my-app-net
[
  {
    "Name": "my-app-net",
    "Id": "b16516e3439a7c8b68472138bf4f81373ca689ef2bc0f2d91a1c4286fe299ead",
    "Created": "2026-03-16T15:22:32.611526402Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "01c2bdf604af14ddb1f1bf00eee8545d071b0e6a15b409c830fdbcb73afee632": {
        "Name": "nginx-1",
        "EndpointID": "8334cd4d1ebbee71e1d87295d54d29c3c3deb2fadb037ceab649383652ad1520",
        "MacAddress": "26:a4:5e:da:1b:a4",
        "IPv4Address": "172.18.0.2/16",
        "IPv6Address": ""
      },
      "0746569eb76c9ac57a524c1c4755d108830659921c5cc0f683b4a8375541f500": {
        "Name": "nginx-2",
        "EndpointID": "abce7f91a34ef6fe383b9152312909aca09579fb21e6b3beeb0633fef17f77f8",
        "MacAddress": "1a:a9:a0:7c:2c:78",
        "IPv4Address": "172.18.0.3/16",
        "IPv6Address": ""
      }
    },
    "Options": {},
    "Labels": {}
  }
]
ubuntu@ip-172-31-30-155:~$
```

Can they ping each other by name now?

```
ubuntu@ip-172-31-30-155:~$ docker exec nginx-1 ping nginx-2
PING nginx-2 (172.18.0.3): 56 data bytes
64 bytes from 172.18.0.3: seq=0 ttl=64 time=0.104 ms
64 bytes from 172.18.0.3: seq=1 ttl=64 time=0.063 ms
64 bytes from 172.18.0.3: seq=2 ttl=64 time=0.067 ms
64 bytes from 172.18.0.3: seq=3 ttl=64 time=0.072 ms
64 bytes from 172.18.0.3: seq=4 ttl=64 time=0.076 ms
64 bytes from 172.18.0.3: seq=5 ttl=64 time=0.099 ms
64 bytes from 172.18.0.3: seq=6 ttl=64 time=0.064 ms
64 bytes from 172.18.0.3: seq=7 ttl=64 time=0.069 ms
64 bytes from 172.18.0.3: seq=8 ttl=64 time=0.066 ms
64 bytes from 172.18.0.3: seq=9 ttl=64 time=0.071 ms
64 bytes from 172.18.0.3: seq=10 ttl=64 time=0.066 ms
^Cubuntu@ip-172-31-30-155:~$ |
```

Write in your notes: Why does custom networking allow name-based communication but the default bridge doesn't?

Default bridge doesn't have a built-in DNS resolver and hence they need IP by default whereas when a custom bridge network is created docker enables a built in DNS resolver and hence it is able to resolve the name to its IP and ping other container in the same network.

Task 6: Put It Together

Create a custom network

```
allen@DESKTOP-CVA040U MINGW64 ~/k8s
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
9c9859c8ed64        bridge              bridge              local
08b8beafa28a        custom-app-net      bridge              local
b7014b10c4b4        host                host                local
72ad3fffe9d5        kind                bridge              local
47584ace1ae2        none                null                local
```

Run a database container (MySQL/Postgres) on that network with a volume for data

Run an app container (use any image) on the same network

```
allen@DESKTOP-CVA040U MINGW64 ~/k8s
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
0c270813e0e4   trainwithshubham/flask-app-ecs:late "python run.py"         7 minutes ago Up 7 minutes   0.0.0.0:80->80/tcp, [::]:80->80/tcp flask-app
1befa868db82   mysql                                "docker-entrypoint.s..." 14 minutes ago Up 14 minutes   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp sql-db
```

Verify the app container can reach the database by container name

```
allen@DESKTOP-CVA040U MINGW64 ~/k8s
$ docker exec flask-app ping sql-db
PING sql-db (172.19.0.2) 56(84) bytes of data.
64 bytes from sql-db.custom-app-net (172.19.0.2): icmp_seq=1 ttl=64 time=2.22 ms
64 bytes from sql-db.custom-app-net (172.19.0.2): icmp_seq=2 ttl=64 time=0.141 ms
64 bytes from sql-db.custom-app-net (172.19.0.2): icmp_seq=3 ttl=64 time=0.103 ms
64 bytes from sql-db.custom-app-net (172.19.0.2): icmp_seq=4 ttl=64 time=0.095 ms
64 bytes from sql-db.custom-app-net (172.19.0.2): icmp_seq=5 ttl=64 time=0.148 ms
64 bytes from sql-db.custom-app-net (172.19.0.2): icmp_seq=6 ttl=64 time=0.072 ms
64 bytes from sql-db.custom-app-net (172.19.0.2): icmp_seq=7 ttl=64 time=0.057 ms
64 bytes from sql-db.custom-app-net (172.19.0.2): icmp_seq=8 ttl=64 time=0.098 ms
64 bytes from sql-db.custom-app-net (172.19.0.2): icmp_seq=9 ttl=64 time=0.099 ms
```